

Bancos de Dados

Aula - 05

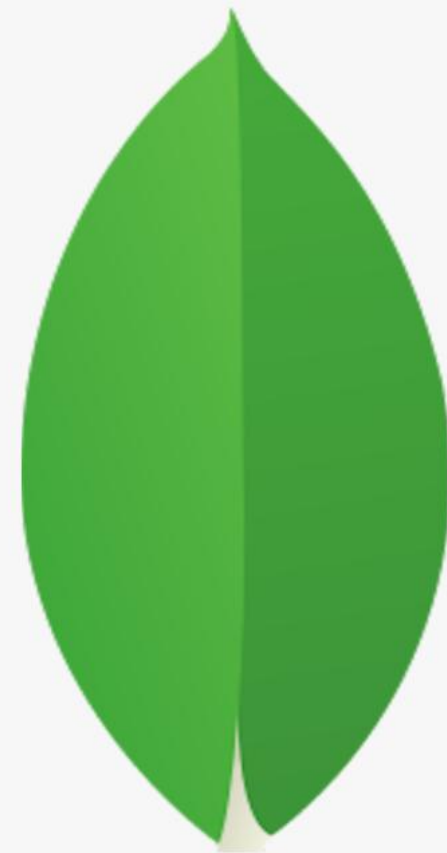
2026

SENAI - CRTI

Operadores de Consulta Avançados no MongoDB

Unidade Curricular: Bancos de Dados

mongoDB®



Roteiro da Aula

01

Situação de Aprendizagem

Contextualização do problema real

02

Mobilização de Conhecimentos

Ativação do que você já sabe

03

Desenvolvimento Teórico

Os 5 operadores avançados com exemplos

04

Prática Progressiva

9 exercícios do básico ao complexo

05

Sistematização e Avaliação

Resumo visual e atividade final



SITUAÇÃO DE APRENDIZAGEM

O Problema Real: Consultas que Não Atendem

O Cenário

Uma aplicação de e-commerce precisa buscar produtos pelo nome parcial, filtrar pedidos de múltiplos clientes ao mesmo tempo e consultar itens dentro de listas complexas de arrays. As consultas básicas com `find()` simples não conseguem atender a esses requisitos.

O Problema

Consultas como `{ nome: "João" }` retornam apenas correspondências exatas. Mas e quando precisamos de:

- Busca parcial por texto ("Jo" encontra "João" e "Jonas")?
- Filtrar vários valores de uma vez?
- Consultar objetos dentro de arrays?
- Verificar se um campo existe no documento?

O Que Você Já Sabe?

Antes de aprender os operadores avançados, vamos ativar os conhecimentos que você já possui. Reflita sobre as perguntas abaixo:

Busca Parcial

Como você buscaria todos os usuários cujo nome **começa com "Jo"**? Uma consulta exata não resolve — o que mais poderia funcionar?

Múltiplos Valores

Se precisar buscar usuários com **cidade "SP" ou "RJ" ou "MG"**, você escreveria três consultas separadas? Existe uma forma mais eficiente?

Dados em Listas

Um pedido contém um array de itens. Como consultar apenas pedidos que contêm **um produto específico com preço acima de R\$2.000**?

 Anote suas respostas agora. Ao final da aula, você verá como os operadores avançados respondem exatamente a essas dúvidas.

Os 5 Operadores Avançados do MongoDB

Cada operador resolve um tipo específico de problema de consulta. Dominar esses operadores transforma a forma como você interage com bancos de dados MongoDB.

`$in`

Busca por múltiplos valores em um campo

`$nin`

Exclui documentos com valores específicos

`$exists`

Verifica se um campo existe no documento

`$regex`

Busca por padrões de texto com expressões regulares

`$elemMatch`

Consulta objetos dentro de arrays com múltiplos critérios

Busca por Múltiplos Valores

O que faz?

O operador `$in` seleciona documentos onde o valor de um campo **corresponde a qualquer um dos valores** de uma lista. É equivalente ao operador `IN` do SQL, mas funciona também com arrays no MongoDB.

Sintaxe

```
{ campo: { $in: [valor1, valor2, ...] } }
```

Exemplo Prático

```
db.usuarios.find({  
  nome: { $in: ["João", "Ana"] }  
});
```

Retorna todos os documentos onde o campo `nome` é **"João"** ou **"Ana"**.

Quando Usar?

- Buscar usuários por uma lista de nomes
- Filtrar produtos por categorias selecionadas
- Consultar pedidos de múltiplos clientes
- Recuperar documentos por lista de IDs

Comparação com SQL

```
-- SQL equivalente:  
SELECT * FROM usuarios  
WHERE nome IN ('João','Ana');
```

Excluindo Valores Específicos

O que faz?

O operador `$nin` (*not in*) é o **inverso do \$in**: retorna documentos cujo campo **não corresponde a nenhum** dos valores da lista. Também retorna documentos onde o campo não existe.

Sintaxe

```
{ campo: { $nin: [valor1, valor2, ...] } }
```

Exemplo Prático

```
db.usuarios.find({
  idade: { $nin: [18, 20] }
});
```

Retorna usuários cuja idade **não é 18 nem 20** anos — ou que não possuem o campo `idade` cadastrado.

Atenção!

⚠ O `$nin` também retorna documentos onde o campo não existe. Combine com `$exists` se quiser garantir que o campo está presente.

Caso de Uso Real

Imagine que você precisa listar produtos de todas as categorias **exceto** "Eletrônicos" e "Informática":

```
db.produtos.find({
  categoria: {
    $nin: ["Eletrônicos", "Informática"]
  }
});
```


Verificando a Existência de Campos



O que faz?

O operador `$exists` verifica se um campo **existe ou não** em um documento. Isso é fundamental no MongoDB, pois diferentemente do SQL, documentos da mesma coleção podem ter **estruturas diferentes**.

Sintaxe

```
{ campo: { $exists: true } }  
{ campo: { $exists: false } }
```

Exemplos

```
// Usuários COM telefone cadastrado  
db.usuarios.find({  
  telefone: { $exists: true }  
});
```

```
// Usuários SEM telefone  
db.usuarios.find({  
  telefone: { $exists: false }  
});
```

Aplicação Prática

Muito útil para identificar documentos **incompletos** ou para migração de dados — encontrar todos os registros que ainda não possuem um campo obrigatório.

Busca por Padrões de Texto

O operador `$regex` permite realizar buscas utilizando **expressões regulares**, possibilitando encontrar documentos com base em padrões de texto — como busca parcial, início ou fim de string, e muito mais.

Sintaxe Básica

```
{ campo: { $regex: "padrão", $options: "flags" } }
```

Flags Importantes

Flag	Significado
<code>i</code>	Case-insensitive (ignora maiúsculas)
<code>m</code>	Multilinha
<code>x</code>	Ignora espaços em branco

Exemplos Práticos

```
// Busca nomes que contêm "Jo" (sem distinção)
db.usuarios.find({
  nome: { $regex: "Jo", $options: "i" }
});
```

```
// Nomes que começam com "Ana"
db.usuarios.find({
  nome: { $regex: "^Ana" }
});
```

```
// Emails que terminam com "@gmail.com"
db.usuarios.find({
  email: { $regex: "@gmail\\.com$" }
});
```

Consultando Arrays de Objetos

O que faz?

O `$elemMatch` permite aplicar **múltiplos critérios sobre um único elemento** de um array de objetos. Sem ele, os critérios podem ser satisfeitos por elementos diferentes do array — o que geralmente não é o que queremos.

Sintaxe

```
{ campo: { $elemMatch: {  
  chave1: valor1,  
  chave2: { $gt: valor2 }  
}  
}}
```

Exemplo: Pedidos com Notebook acima de R\$2.000

```
db.pedidos.find({  
  itens: {  
    $elemMatch: {  
      produto: "Notebook",  
      preco: { $gt: 2000 }  
    }  
  }  
});
```

 Sem o `$elemMatch`, o MongoDB poderia retornar pedidos onde "Notebook" e "preco > 2000" são de itens diferentes no array. O `$elemMatch` garante que AMBOS os critérios se aplicam ao MESMO elemento.

Consultas Complexas com Múltiplos Operadores

A verdadeira potência do MongoDB aparece quando **combinamos operadores** em uma única consulta. Isso permite criar filtros extremamente precisos e eficientes.

```
// Buscar usuários chamados "João" ou "Ana",  
// que NÃO tenham 18 ou 20 anos,  
// e que possuam o campo "telefone"  
db.usuarios.find(  
  nome:      { $in: ["João", "Ana"] },  
  idade:     { $nin: [18, 20] },  
  telefone: { $exists: true }  
});
```

Dica de Performance

Coloque os filtros mais **restritivos primeiro**. O MongoDB avalia os critérios em ordem, então começar pelo campo que elimina mais documentos torna a consulta mais rápida.

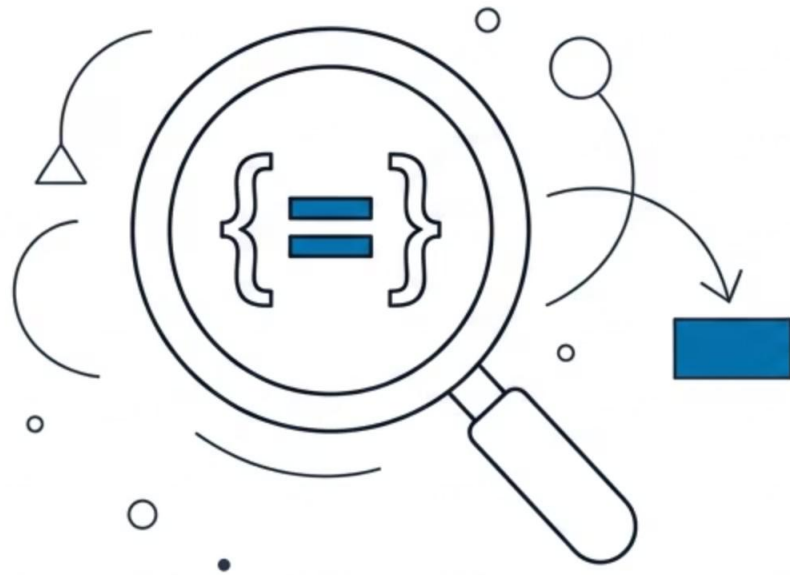
Legibilidade

Use **indentação e comentários** ao escrever consultas complexas. Em produção, consultas bem formatadas são mais fáceis de manter e depurar.

Consulta Simples vs. Consulta Avançada

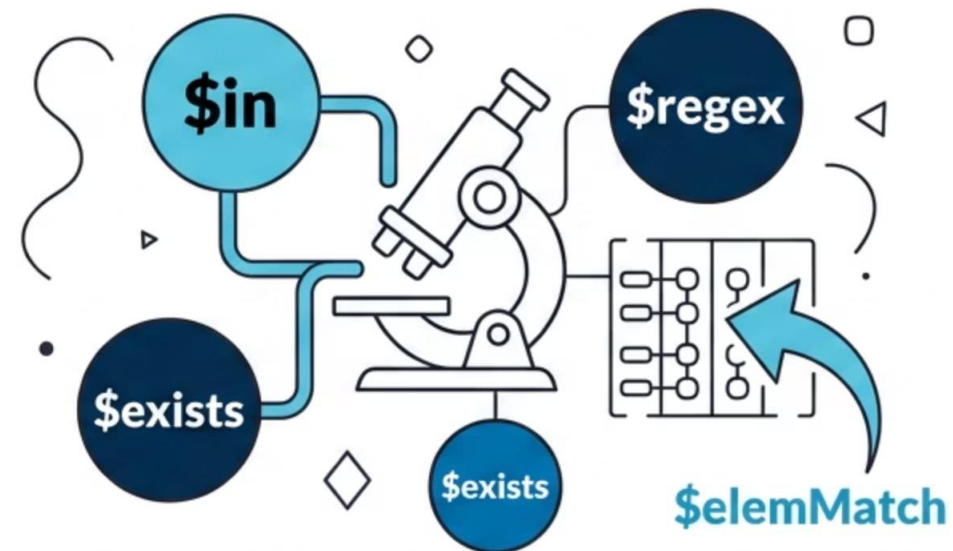
Entender a diferença entre o que as consultas básicas fazem e o que os operadores avançados permitem é fundamental para escolher a ferramenta certa.

CONSULTA SIMPLES



- IGUALDADE EXATA
- RETORNA UM VALOR
- NÃO SUPORTA PADRÕES DE TEXTO
- NÃO VERIFICA ARRAYS

CONSULTA AVANÇADA



- \$IN: MÚLTIPLOS VALORES
- \$REGEX: PADRÕES
- \$EXISTS: VERIFICAÇÃO DE CAMPOS
- \$ELEMATCH: ARRAYS DE OBJETOS

Estrutura dos Exercícios Práticos

Os exercícios foram organizados em **ordem crescente de complexidade**, seguindo a lógica de construção gradual de competências da MSEP. Você vai do simples ao complexo, com cada exercício preparando o terreno para o próximo.

1

Nível 1

Exercícios 1–3

Operadores isolados

2

Nível 2

Exercícios 4–6

Texto e arrays simples

3

Nível 3

Exercícios 7–8

Arrays de objetos e combinações

4

Nível 4

Exercício 9

Cenário real completo

📌 Use o MongoDB Compass ou o terminal mongosh para executar cada exercício. Anote os resultados obtidos — eles serão importantes na sistematização.

EXERCÍCIO 1

Busca com \$in

Contexto

Você tem uma coleção `usuarios` com documentos contendo campos `nome`, `idade`, `cidade` e `telefone`. O sistema precisa listar usuários de cidades específicas.

Tarefa

Escreva uma consulta usando `$in` para buscar todos os usuários que moram em **"São Paulo"**, **"Rio de Janeiro"** ou **"Belo Horizonte"**.

Gabarito

```
db.usuarios.find({
  cidade: {
    $in: [
      "São Paulo",
      "Rio de Janeiro",
      "Belo Horizonte"
    ]
  }
});
```

Verifique

- Quantos documentos foram retornados?
- Algum usuário de outra cidade apareceu?
- O que acontece se você adicionar uma cidade inexistente à lista?

Filtrando com \$nin e Verificando com \$exists

Exercício 2 — \$nin

Liste todos os usuários cuja **idade não seja 18 nem 20** anos. Em seguida, experimente excluir também a idade 25.

```
db.usuarios.find({
  idade: { $nin: [18, 20] }
});
```

⚠ Observe se aparecem documentos sem o campo "idade". Isso é comportamento esperado do \$nin!

Exercício 3 — \$exists

Encontre todos os usuários que **possuem o campo telefone** cadastrado. Depois, modifique a consulta para encontrar quem **não tem** telefone.

```
// Com telefone
db.usuarios.find({
  telefone: { $exists: true }
});

// Sem telefone
db.usuarios.find({
  telefone: { $exists: false }
});
```

Compare os resultados: a soma dos dois grupos deve ser igual ao total de documentos na coleção.

Buscas por Padrão de Texto com \$regex

Exercício 4 — Busca Parcial

Busque todos os usuários cujo nome **contenha "Jo"**, independentemente de maiúsculas ou minúsculas. Isso deve retornar "João", "Jonas", "jose", etc.

```
db.usuarios.find({
  nome: {
    $regex: "Jo",
    $options: "i"
  }
});
```

Exercício 5 — Início de String

Agora refine a busca: encontre apenas usuários cujo nome **começa com "Ana"**. Use o símbolo `^` para indicar início da string.

```
db.usuarios.find({
  nome: { $regex: "^Ana" }
});
```

Desafio Extra

Escreva uma consulta que encontre usuários cujo email **termina com "@gmail.com"**. Use `$` para indicar final da string.

```
db.usuarios.find({
  email: {
    $regex: "@gmail\\.com$",
    $options: "i"
  }
});
```

Consultando Arrays Simples

Contexto

Uma coleção `produtos` possui um campo `tags` que é um array de strings. Por exemplo: `["eletrônico", "portátil", "oferta"]`. O MongoDB permite consultar arrays diretamente, sem operadores especiais para casos simples.

Consulta Direta em Array

```
// Buscar produtos com a tag
"oferta"
db.produtos.find({
  tags: "oferta"
});
```

```
// Buscar produtos com QUALQUER
tag da lista
db.produtos.find({
  tags: { $in: ["portátil",
"oferta"] }
});
```

Reflexão

Quando o valor de um campo é um array e você consulta com igualdade simples, o MongoDB verifica se **algum elemento** do array corresponde ao valor. Isso é diferente de exigir que **todos** os critérios sejam satisfeitos pelo **mesmo elemento** — para isso, use `$elemMatch`.

```
{ } 1  var engage = {
2      last_login : "September 13, 2012",
3      section_updates : [
4          {
5              name : "buzz",
6              date: "September 13, 2012"
7          },
8          {
9              name : "opportunities",
10             date: "September 13, 2012"
11          }
12      ],
13      impressions : [
14          {
15              user      : "",
16              unique    : "c980bcedcb58e5a874ce1b5dc8c5c0ab",
17              time      : "September 13, 2012 11:13:00",
18          },
19          {
20              user      : "",
21              unique    : "c980bcedcb58e5a874ce1b5dc8c5c0ab",
22              time      : "September 13, 2012 11:13:00",
23          },
24          {
25              user      : "",
26              unique    : "c980bcedcb58e5a874ce1b5dc8c5c0ab",
27              time      : "September 13, 2012 11:13:00",
28          },
29          {
30              user      : "",
31              unique    : "c980bcedcb58e5a874ce1b5dc8c5c0ab",
32              time      : "September 13, 2012 11:13:00",
33          },
34          {
35              user      : "",
36              unique    : "c980bcedcb58e5a874ce1b5dc8c5c0ab",
37              time      : "September 13, 2012 11:13:00",
38          },
39          {
40              user      : "",
41              unique    : "c980bcedcb58e5a874ce1b5dc8c5c0ab",
42              time      : "September 13, 2012 11:13:00",
43          }
44      ],
45  }
```

EXERCÍCIO 7

Consultando Arrays de Objetos com \$elemMatch

Este é um dos exercícios mais importantes da aula. O `$elemMatch` garante que os critérios se apliquem ao **mesmo elemento** do array.

Estrutura de um Pedido

```
{
  _id: ObjectId("..."),
  cliente: "Maria",
  itens: [
    { produto: "Notebook",
      preco: 3500 },
    { produto: "Mouse",
      preco: 80 }
  ]
}
```

Tarefa

Busque pedidos que contenham um item com **produto "Notebook" e preço acima de R\$2.000** — ambos no mesmo elemento do array.

```
db.pedidos.find({
  itens: {
    $elemMatch: {
      produto: "Notebook",
      preco: { $gt: 2000 }
    }
  }
});
```

 Teste também sem o `$elemMatch` para ver a diferença no resultado. Você entenderá porque o operador é tão importante!

Consultas Combinadas com Múltiplos Operadores

Chegou a hora de combinar tudo o que aprendemos! Consultas reais raramente usam apenas um operador — elas combinam vários para atingir a precisão necessária.

Desafio

Crie uma consulta que retorne usuários que satisfaçam **todas** as condições abaixo simultaneamente:

- Nome começa com "J" ou "A"
- Idade não é 18 nem 20
- Possui campo telefone cadastrado

Solução

```
db.usuarios.find({
  nome: {
    $regex: "^[JA]",
    $options: "i"
  },
  idade: { $nin: [18, 20] },
  telefone: { $exists: true }
});
```

Como Funciona?

O MongoDB aplica todos os critérios com lógica **AND implícito**: somente documentos que satisfazem **todos** os campos simultaneamente são retornados. Para OR explícito, usaríamos o operador `$or`.

EXERCÍCIO 9

Cenário Real: Sistema de E-Commerce

Aplique os operadores avançados em um cenário completo de e-commerce com as coleções **produtos** e **pedidos**.

1

Produtos em Promoção

Busque produtos das categorias **"Smartphone"** ou **"Tablet"** cujo nome contenha **"Pro"** e que tenham o campo **desconto** existente.

2

Pedidos de Alto Valor

Encontre pedidos com algum item cujo preço seja **acima de R\$1.500**, excluindo itens da categoria **"Acessórios"**.

3

Clientes Prioritários

Encontre clientes cujo nome **começa com "R"**, que **não são** do nível "bronze", e que possuam endereço cadastrado (**endereco: \$exists: true**).



Resumo Visual dos Operadores

Veja o mapa completo dos operadores estudados, com suas finalidades, sintaxes e casos de uso — guarde esta referência!

Operador	O que faz	Exemplo	Quando usar
<code>\$in</code>	Busca por múltiplos valores	<code>{ cidade: { \$in: ["SP","RJ"] } }</code>	Filtrar por lista de opções
<code>\$nin</code>	Exclui valores específicos	<code>{ idade: { \$nin: [18,20] } }</code>	Descartar categorias ou faixas
<code>\$exists</code>	Verifica existência de campo	<code>{ tel: { \$exists: true } }</code>	Dados incompletos ou opcionais
<code>\$regex</code>	Busca por padrão de texto	<code>{ nome: { \$regex: "^Jo" } }</code>	Busca parcial, autocompletar
<code>\$elemMatch</code>	Critérios em mesmo elemento de array	<code>{ itens: { \$elemMatch: {...} } }</code>	Arrays de objetos complexos

Quando Usar Cada Operador?



Use este guia de decisão para escolher o operador certo. Na prática, a maioria das consultas reais combina dois ou mais operadores para criar filtros precisos e eficientes.

Atividade Prática Individual — 30 Minutos

Situação

Você é desenvolvedor(a) back-end em uma startup de delivery. O banco de dados possui coleções `clientes`, `restaurantes` e `pedidos`. Você precisa criar consultas para o painel administrativo.

Entregável

Um arquivo `.js` ou `.txt` com todas as consultas escritas, comentadas e com os resultados descritos.

Tarefas

1. Buscar clientes de "Curitiba", "Florianópolis" ou "Porto Alegre" usando `$in`
2. Listar restaurantes que **não** são da categoria "FastFood" ou "Pizza" com `$nin`
3. Encontrar pedidos que possuam o campo `avaliacao` preenchido com `$exists`
4. Buscar restaurantes cujo nome contenha "Bistr" com `$regex`
5. Encontrar pedidos com item de preço acima de R\$50 usando `$elemMatch`
6. Criar uma consulta combinada com pelo menos 3 operadores diferentes



